

STOCKIQ · DOCUMENTAÇÃO DO BACKEND

Backend API REST

Documentação técnica da API do StockIQ: arquitetura, modelo de dados, regras de negócio, segurança e catálogo completo de endpoints.

Node.js · Express

TypeScript

PostgreSQL · Prisma

JWT · Zod

75 endpoints

Gabriel da Silveira Pessoni · Lívia Portela Ferreira

FATEC Franca — Desenvolvimento de Software Multiplataforma ·
6º semestre

Projeto Integrador VI

Computação em Nuvem II · Mineração
de Dados

Sumário

01	Visão geral	Escopo, números e responsabilidades
02	Stack tecnológica	Tecnologias e finalidade
03	Arquitetura em camadas	Rotas, controllers, services, Prisma
04	Ciclo de vida da requisição	Pipeline de middlewares
05	Modelo de dados	ER, entidades e enumerações
06	Autenticação e autorização	JWT, RBAC e matriz de permissões
07	Regras de negócio	Movimentações, alertas, lotes, ABC
08	Catálogo de endpoints	15 módulos · 75 rotas
09	Validação e tratamento de erros	Zod, AppError, códigos HTTP
10	Segurança e auditoria	Helmet, CORS, rate limit, audit log
11	Testes automatizados	Jest + Supertest
12	Operação e execução	Variáveis, scripts, Swagger
13	Infraestrutura em nuvem	AWS, SQS e pipeline de dados

01 Visão geral

O backend do StockIQ é uma API REST em Node.js/Express com TypeScript que concentra toda a regra de negócio, a persistência em PostgreSQL (via Prisma ORM) e a autenticação do sistema. É consumida pelo front-end Next.js por HTTP/JSON.

15

MÓDULOS

75

ENDPOINTS

12

ENTIDADES

11

ENUMERAÇÕES

A API cobre o ciclo completo de operação de um armazém: produtos, categorias, marcas, fornecedores e clientes; endereçamento físico; movimentações de estoque (entrada, saída, transferência, perda, ajuste e inventário); lotes com controle de validade; contagens de inventário; alertas automáticos; auditoria; dashboard e relatórios gerenciais, incluindo curva ABC.

**Segurança**

JWT, bcrypt, Helmet, CORS e rate limiting em camadas.

**Validação**

Schemas Zod por módulo antes de qualquer regra de negócio.

**Camadas claras**

routes → controllers → services
→ Prisma, sem vazamento de responsabilidades.

**Documentada**

Swagger UI e OpenAPI 3.0 gerados a partir das rotas.

**Testada**

Testes de integração com Jest + Supertest nos fluxos principais.

**Analítica**

Dashboard, relatórios e curva ABC prontos para mineração de dados.

02 Stack tecnológica

Camada	Tecnologia	Finalidade
Linguagem	TypeScript 5	Tipagem estática ponta a ponta; reduz erros em tempo de execução
Framework HTTP	Express 4	Roteamento e cadeia de middlewares
ORM	Prisma 6	Modelagem, migrations versionadas e acesso tipado ao PostgreSQL
Banco de dados	PostgreSQL	Persistência relacional transacional
Autenticação	jsonwebtoken · bcryptjs	Token JWT assinado e hash de senha (custo 10)
Validação	Zod 3	Schemas de entrada com mensagens de erro padronizadas
Segurança	Helmet · CORS · express-rate-limit	Cabeçalhos seguros, controle de origem e limite de requisições
Documentação	swagger-jsdoc · swagger-ui-express	OpenAPI 3.0 em <code>/docs</code> e <code>/docs.json</code>
Testes	Jest 29 · Supertest · ts-jest	Testes de integração dos endpoints
Logs	Morgan	Log de requisições HTTP (silenciado em ambiente de teste)

03 Arquitetura em camadas

Cada módulo de negócio é composto por quatro artefatos com responsabilidades distintas. Preocupações transversais (autenticação, validação, auditoria, limites e erros) vivem em middlewares e nunca dentro de controllers ou services.

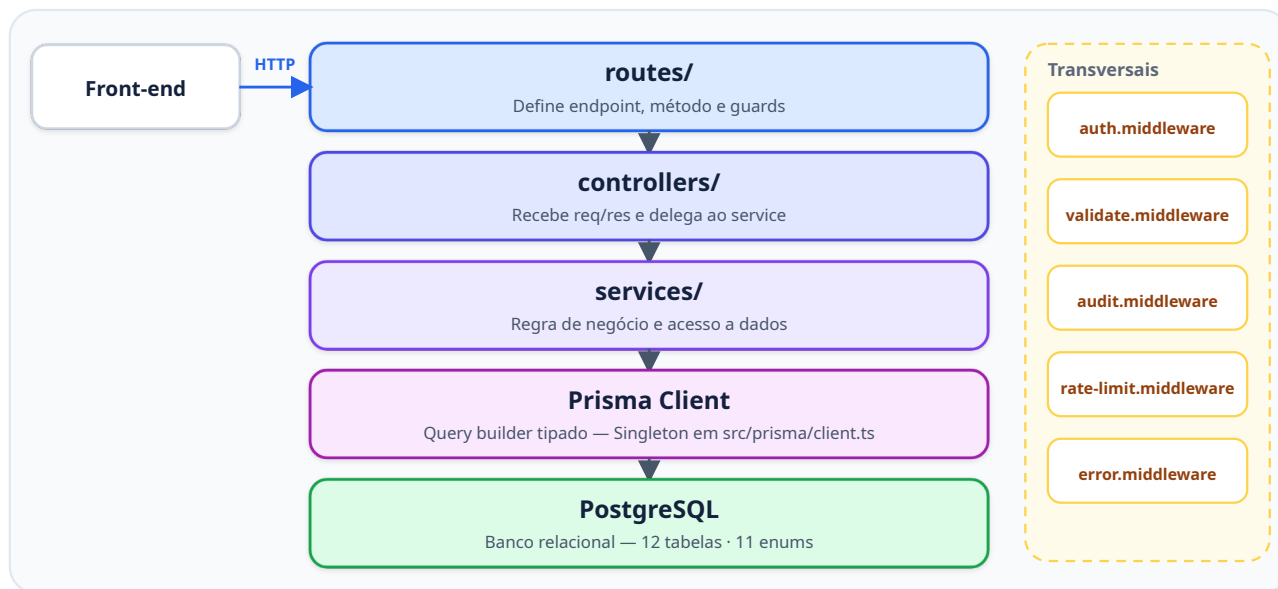


Figura 1 — Camadas da API e middlewares transversais

● Estrutura de pastas

```
backend/
├── src/
│   ├── app.ts                # Express, middlewares globais e registro de rotas
│   ├── server.ts             # Bootstrap: conecta ao banco e sobe o HTTP
│   ├── config/swagger.ts     # Definição OpenAPI
│   ├── routes/ (15)          # Endpoints + guards de acesso
│   ├── controllers/ (12)     # req/res
│   ├── services/ (14)        # Regra de negócio + Prisma
│   ├── schemas/ (9)          # Schemas Zod de entrada
│   ├── middleware/           # auth · validate · audit · error · rate-limit
│   └── prisma/client.ts      # Prisma Client singleton
├── prisma/
│   ├── schema.prisma         # Modelo de dados
│   ├── seed.ts               # Massa de demonstração
│   └── migrations/           # Histórico versionado
├── __tests__/                 # Testes de integração (Jest + Supertest)
└── .env.example
```

04 Ciclo de vida da requisição

Toda requisição atravessa uma cadeia fixa de middlewares. Qualquer falha é convertida em uma resposta JSON padronizada pelo manipulador de erros central.

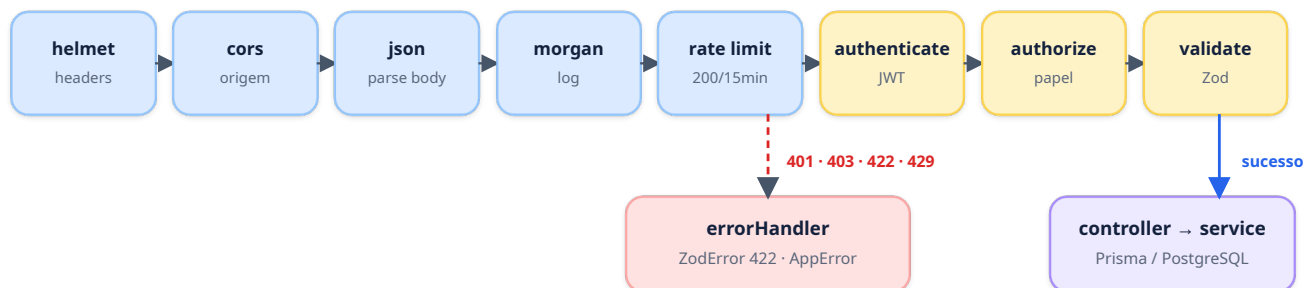


Figura 2 — Pipeline de middlewares (azul: globais · amarelo: por rota)

● Sequência — registro de uma movimentação

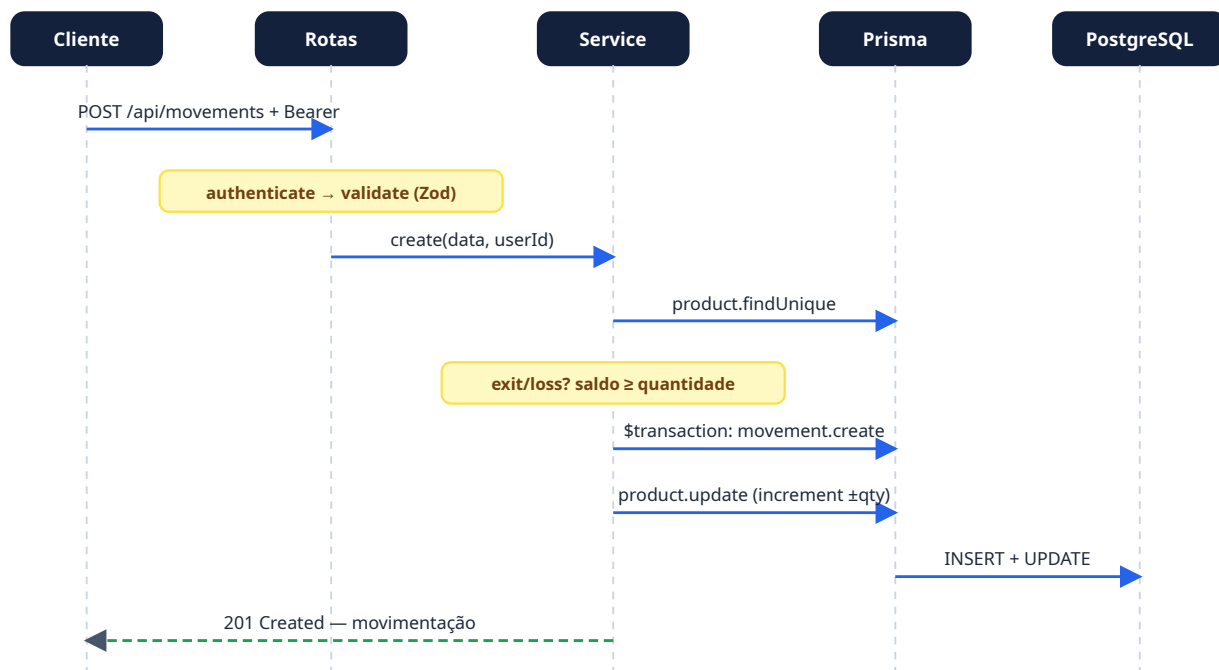


Figura 3 — Diagrama de sequência: POST /api/movements

05 Modelo de dados

O modelo é definido em `prisma/schema.prisma` e versionado por migrations. São 12 entidades e 11 enumerações; todos os identificadores são `cuid` e as tabelas usam nomes em snake_case plural.

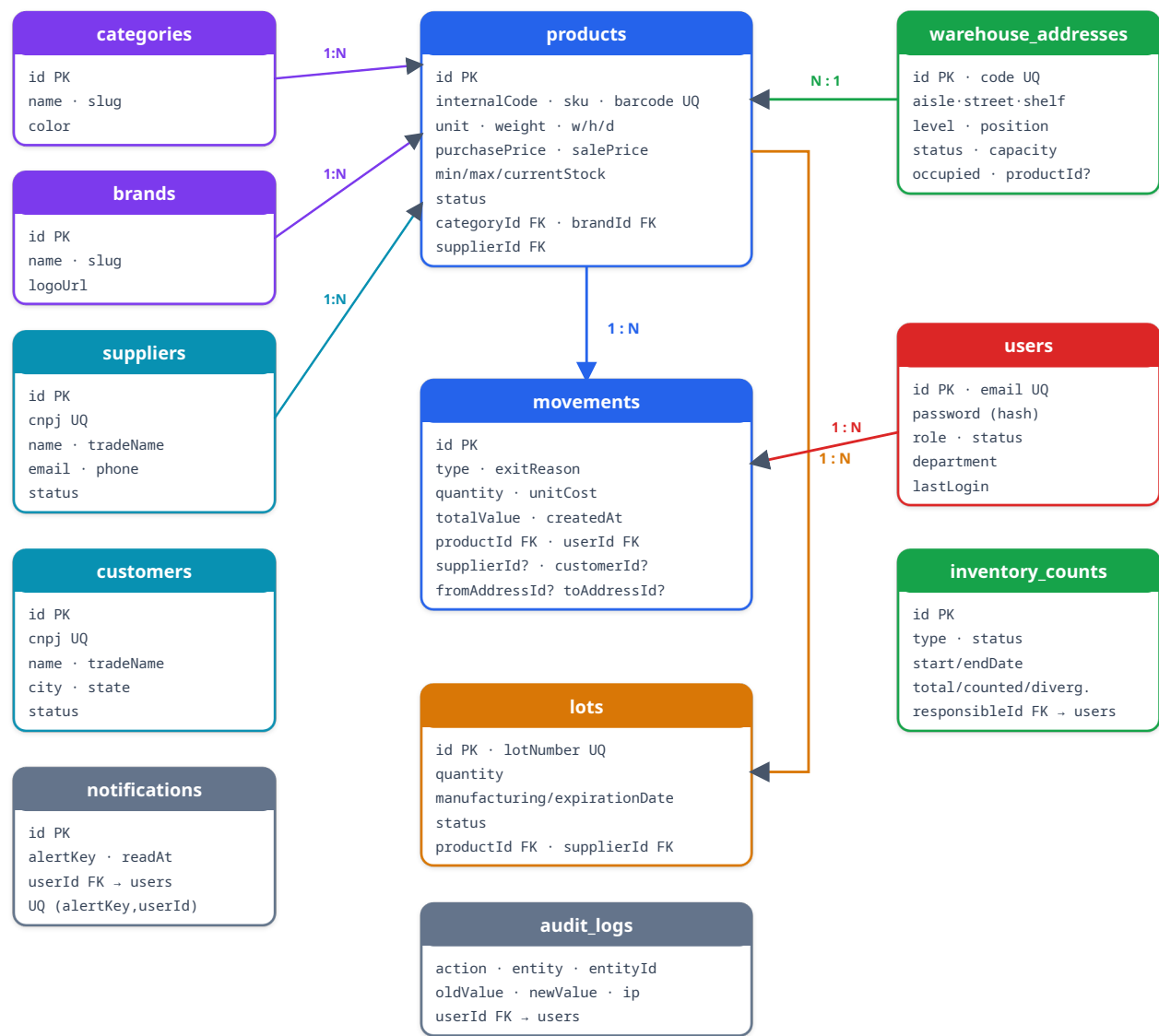


Figura 4 — Diagrama entidade-relacionamento (simplificado)

Entidades

Entidade	Campos principais	Relacionamentos
User	name, email (único), password (hash), role, department, status, avatarUrl, lastLogin	1:N Movement, AuditLog, InventoryCount, Notification
Category	name, slug, color	1:N Product
Brand	name, slug, logoUrl	1:N Product

Entidade	Campos principais	Relacionamentos
Supplier	name, tradeName, cnpj, email, phone, contactName, category, status, city, state	1:N Product, Movement, Lot
Customer	name, tradeName, cnpj, email, phone, contactName, city, state, status	Referenciado por Movement
Product	name, internalCode, sku, barcode, unit, dimensões, preços, min/max/currentStock, status	N:1 Category, Brand, Supplier; 1:N Movement, Lot, WarehouseAddress
WarehouseAddress	code, aisle, street, shelf, level, position, status, capacity, occupied, lotNumber, quantity	N:1 Product (opcional); origem/destino de Movement
Movement	type, quantity, unitCost, totalValue, invoiceNumber, lotNumber, exitReason, notes, createdAt	N:1 Product, User; opcionais Supplier, endereços
Lot	lotNumber (único), quantity, manufacturingDate, expirationDate, address, status	N:1 Product, Supplier
InventoryCount	name, type, status, startDate, endDate, totalItems, countedItems, divergences	N:1 User (responsável)
Notification	alertKey, readAt — único por (alertKey, userId)	N:1 User
AuditLog	action, entity, entityId, entityName, oldValue, newValue (JSON), ip	N:1 User

Enumerações

Enum	Valores
UserRole	admin · supervisor · operator · viewer
UserStatus	active · inactive · pending
ProductStatus	active · inactive · discontinued
PositionStatus	free · occupied · blocked · reserved
MovementType	entry · exit · transfer · loss · adjustment · inventory
ExitReason	sale · transfer · loss · break · internal
LotStatus	valid · expiring · expired · quarantine
InventoryCountType	full · partial · cyclic
InventoryCountStatus	planned · in_progress · review · completed
SupplierStatus · CustomerStatus	active · inactive

06 Autenticação e autorização

O login retorna um token JWT assinado com `JWT_SECRET`, contendo `sub` (id), `email` e `role`. Rotas protegidas exigem `Authorization: Bearer <token>`. Senhas são armazenadas apenas como hash bcrypt.

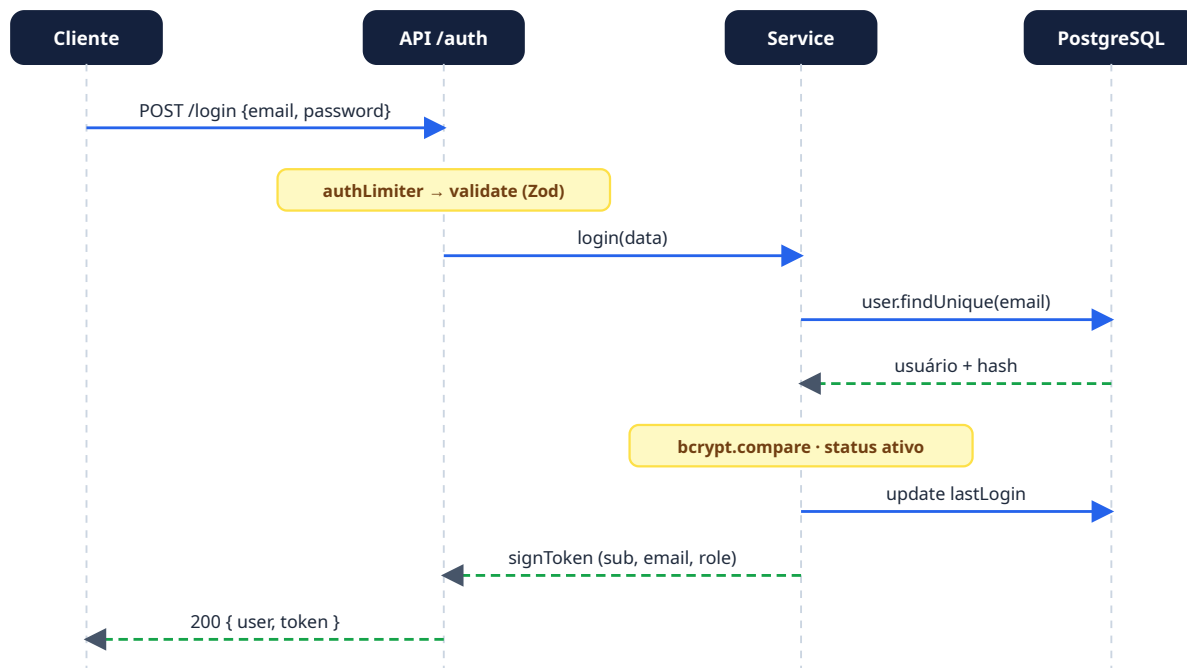


Figura 5 — Fluxo de login

■ Hierarquia de papéis

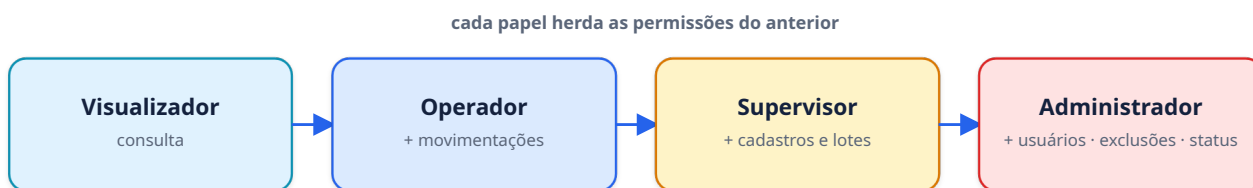


Figura 6 — Hierarquia de acesso (RBAC)

● Matriz de permissões por módulo

Módulo	Leitura	Criar	Alterar	Excluir
Produtos · Categorias · Marcas · Fornecedores · Clientes · Endereços	Autenticado	Supervisor+	Supervisor+	Admin
Usuários	Autenticado	Supervisor+	Supervisor+ (status: Admin)	Admin
Movimentações	Autenticado	Autenticado	—	—
Lotes	Autenticado	Operador+	Supervisor+	—
Inventário (contagens)	Autenticado	Supervisor+	Operador+	Admin
Dashboard · Alertas · Relatórios	Autenticado	—	marcar alerta como lido: Autenticado	—
Auditoria	Supervisor+	—	—	—
Auth: register · login	Público	—	senha: Autenticado	—

COMO É APLICADO

`authenticate` valida o token e popula `req.user`. `authorize('admin','supervisor')` compara `req.user.role` com a lista permitida (as listas já incluem os papéis superiores). Falhas retornam 401 (token) ou 403 (permissão).

07 Regras de negócio

● Registro de movimentações

A criação de uma movimentação é atômica: o registro do movimento e a atualização do saldo do produto ocorrem na mesma transação do banco.



Figura 7 — Fluxo de decisão ao registrar uma movimentação

O valor total é calculado como `quantity × unitCost`. Os tipos `entry` e `adjustment` incrementam o saldo; os demais o decrementam.

● Situação de estoque

Situação	Regra (saldo × mínimo)	Uso
ok	saldo > mínimo	Listagem de produtos e filtros
low estoque baixo	mínimo/2 < saldo ≤ mínimo	Alerta de reposição
critical crítico	0 < saldo ≤ mínimo/2	Alerta de alta prioridade
out sem estoque	saldo = 0	Alerta de ruptura

● Alertas

O módulo de alertas consolida, em tempo real, os produtos em ruptura, críticos e com estoque baixo, além de lotes a vencer (janela padrão de 30 dias) e lotes vencidos. A leitura por usuário é persistida na tabela `notifications` por `alertKey`, de forma que cada pessoa mantém seu próprio estado de “lido”.

● Curva ABC

Os produtos são ordenados pelo valor total movimentado (`Σ totalValue`) e classificados pelo percentual acumulado:

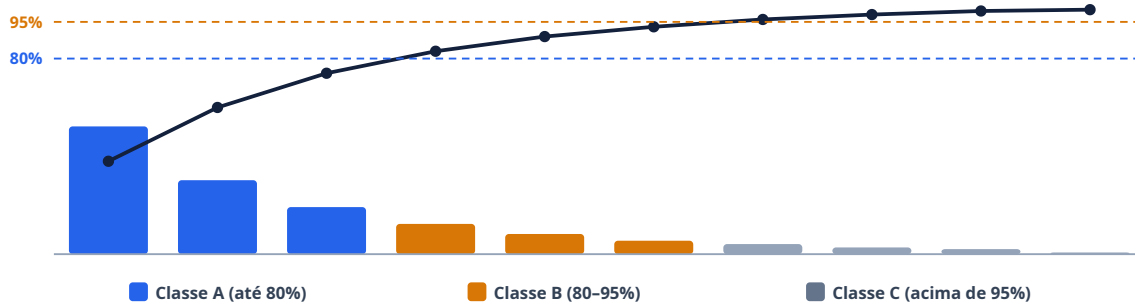


Figura 8 — Ilustração da classificação ABC (valores ilustrativos)

● Lotes e validade

Cada lote possui número único, data de fabricação e validade, endereço e status (`valid` , `expiring` , `expired` , `quarantine`). A listagem aceita filtros por produto, status e janela de vencimento (`expiringSoonDays`), sempre ordenada pela data de validade.

● Dashboard

Endpoint	Conteúdo
<code>GET /api/dashboard</code>	Totais de produtos (ativos, estoque baixo, sem estoque), movimentações do dia, usuários ativos, ocupação do armazém, valor de estoque (compra e venda) e 10 movimentações recentes
<code>/movement-trend</code>	Série mensal de entradas, saídas e saldo (12 meses por padrão)
<code>/category-distribution</code>	Distribuição do estoque por categoria
<code>/top-products</code>	Produtos mais movimentados
<code>/abc</code>	Curva ABC em classes A, B e C

08 Catálogo de endpoints

Base URL padrão: `http://localhost:3001`. Todas as rotas abaixo ficam sob `/api`. Listagens aceitam paginação (`page`, `limit` ≤ 100) e retornam `{ data, total, page, limit, totalPages }`.

01 · Autenticação `/api/auth`

Método	Rota	Descrição	Acesso
POST	<code>/register</code>	Registrar usuário (papel padrão: operador)	Público
POST	<code>/login</code>	Login e emissão do token	Público
GET	<code>/me</code>	Dados do usuário autenticado	Autenticado
GET	<code>/profile</code>	Perfil completo	Autenticado
PATCH	<code>/password</code>	Alterar a própria senha	Autenticado

02 · Usuários `/api/users`

Método	Rota	Descrição	Acesso
GET	<code>/</code>	Listar (paginado)	Autenticado
GET	<code>/:id</code>	Buscar por id	Autenticado
POST	<code>/</code>	Criar usuário	Supervisor+
PATCH	<code>/:id</code>	Atualizar usuário	Supervisor+
PATCH	<code>/:id/status</code>	Alterar status	Admin
DELETE	<code>/:id</code>	Remover usuário	Admin

03 · Produtos `/api/products`

Método	Rota	Descrição	Acesso
GET	<code>/scan/:code</code>	Buscar por código de barras / SKU / código interno	Autenticado
GET	<code>/</code>	Listar com busca, categoria, marca, status e situação de estoque	Autenticado
GET	<code>/:id</code>	Detalhe com situação de estoque	Autenticado
POST	<code>/</code>	Criar produto (SKU e código únicos)	Supervisor+
PATCH	<code>/:id</code>	Atualizar produto	Supervisor+
DELETE	<code>/:id</code>	Remover produto	Admin

04 · Categorias `/api/categories`

Método	Rota	Descrição	Acesso
GET	/	Listar	Autenticado
GET	/:id	Buscar por id	Autenticado
POST	/	Criar	Supervisor+
PATCH	/:id	Atualizar	Supervisor+
DELETE	/:id	Remover	Admin

05 · Marcas `/api/brands`

Método	Rota	Descrição	Acesso
GET	/	Listar	Autenticado
GET	/:id	Buscar por id	Autenticado
POST	/	Criar	Supervisor+
PATCH	/:id	Atualizar	Supervisor+
DELETE	/:id	Remover	Admin

06 · Fornecedores `/api/suppliers`

Método	Rota	Descrição	Acesso
GET	/	Listar (paginado)	Autenticado
GET	/:id	Buscar por id	Autenticado
POST	/	Criar	Supervisor+
PATCH	/:id	Atualizar	Supervisor+
DELETE	/:id	Remover	Admin

07 · Clientes /api/customers

Método	Rota	Descrição	Acesso
GET	/	Listar com busca e paginação	Autenticado
GET	/:id	Buscar por id	Autenticado
POST	/	Criar	Supervisor+
PATCH	/:id	Atualizar	Supervisor+
DELETE	/:id	Remover	Admin

08 · Armazém (endereços) /api/warehouse

Método	Rota	Descrição	Acesso
GET	/stats	Estatísticas de ocupação	Autenticado
GET	/	Listar endereços (filtros)	Autenticado
GET	/:id	Buscar por id	Autenticado
POST	/	Criar endereço	Supervisor+
PATCH	/:id	Atualizar	Supervisor+
DELETE	/:id	Remover	Admin

09 · Movimentações /api/movements

Método	Rota	Descrição	Acesso
GET	/	Listar por tipo, produto e período	Autenticado
GET	/:id	Buscar por id	Autenticado
POST	/	Registrar entrada, saída, transferência, perda, ajuste ou inventário	Autenticado

10 · Lotes /api/lots

Método	Rota	Descrição	Acesso
GET	/alerts	Lotes a vencer / vencidos	Autenticado
GET	/	Listar (produto, status, vencimento)	Autenticado
GET	/:id	Buscar por id	Autenticado
POST	/	Criar lote (número único)	Operador+
PATCH	/:id	Atualizar lote	Supervisor+

11 · Inventário `/api/inventory`

Método	Rota	Descrição	Acesso
GET	/	Listar contagens	Autenticado
GET	/:id	Detalhe da contagem	Autenticado
POST	/	Planejar contagem	Supervisor+
PATCH	/:id	Atualizar andamento	Operador+
DELETE	/:id	Remover contagem	Admin

12 · Dashboard `/api/dashboard`

Método	Rota	Descrição	Acesso
GET	/	Resumo geral	Autenticado
GET	/movement-trend	Tendência mensal	Autenticado
GET	/category-distribution	Distribuição por categoria	Autenticado
GET	/top-products	Mais movimentados	Autenticado
GET	/abc	Curva ABC	Autenticado

13 · Alertas `/api/alerts`

Método	Rota	Descrição	Acesso
GET	/	Todos os alertas com estado de leitura	Autenticado
GET	/stock	Alertas de estoque por severidade	Autenticado
GET	/expiring	Lotes próximos ao vencimento	Autenticado
PATCH	/read-all	Marcar todos como lidos	Autenticado
PATCH	/:id/read	Marcar um alerta como lido	Autenticado

14 · Auditoria `/api/audit`

Método	Rota	Descrição	Acesso
GET	/	Logs de auditoria	Supervisor+
GET	/:id	Detalhe de um log	Supervisor+

15 · Relatórios `/api/reports`

Método	Rota	Descrição	Acesso
GET	<code>/movements</code>	Movimentações por tipo e produto	Autenticado
GET	<code>/stock</code>	Posição de estoque valorizada	Autenticado
GET	<code>/lots</code>	Validade de lotes	Autenticado
GET	<code>/warehouse</code>	Ocupação do armazém	Autenticado
GET	<code>/abc</code>	Curva ABC	Autenticado
GET	<code>/inventory</code>	Contagens e divergências	Autenticado
GET	<code>/suppliers</code>	Compras por fornecedor	Autenticado

09 Validação e tratamento de erros

Cada módulo possui schemas Zod em `src/schemas`. O middleware `validate` valida o corpo e `validateQuery` valida e converte (`coerce`) os parâmetros de consulta antes de chegarem ao controller.

```
const createMovementSchema = z.object({
  type: z.enum(['entry', 'exit', 'transfer', 'loss', 'adjustment', 'inventory']),
  productId: z.string().cuid(),
  quantity: z.number().int().positive(),
  unitCost: z.number().min(0).default(0),
  exitReason: z.enum([...]).optional(),
  ...
})
```

● Padrão de respostas de erro

Código	Origem	Corpo
401	Token ausente, inválido ou expirado; credenciais inválidas	{ message }
403	Papel sem permissão; conta inativa	{ message }
404	Recurso não encontrado	{ message }
409	Conflito de unicidade (e-mail, SKU, número de lote...)	{ message }
400	Regra de negócio violada (ex.: estoque insuficiente)	{ message }
422	Falha de validação Zod	{ message, errors: [{ field, message }] }
429	Limite de requisições excedido	{ message }
500	Erro inesperado (detalhes apenas no log do servidor)	{ message }

10 Segurança e auditoria

Mecanismo	Detalhes
Helmet	Cabeçalhos HTTP de segurança em toda a API
CORS	Habilitado para consumo pelo front-end
Rate limiting	<code>globalLimiter</code> : 200 req / 15 min por IP em toda a API · <code>authLimiter</code> : 20 req / 15 min em <code>/auth/login</code> e <code>/auth/register</code>
Senhas	Hash bcrypt (custo 10); o hash nunca é devolvido nas respostas
JWT	Assinado com <code>JWT_SECRET</code> , expiração configurável (<code>JWT_EXPIRES_IN</code>)
Validação	Toda entrada passa por schema Zod antes da regra de negócio
Erros	Classe <code>AppError</code> + handler central; erros internos não vazam detalhes ao cliente

● Auditoria

A tabela `audit_logs` registra **ação, entidade, id e nome da entidade, valor novo/antigo (JSON), IP e usuário**. O middleware `audit(entity, action)` intercepta a resposta JSON de operações bem-sucedidas e grava o log de forma assíncrona, sem atrasar a resposta. A consulta é feita em `/api/audit`, restrita a supervisor e administrador.

11 Testes automatizados

A suíte de integração (Jest + Supertest) exercita a aplicação Express real contra o banco, cobrindo autenticação, controle de acesso, validação e regras de negócio dos módulos centrais.

Arquivo	Escopo	Casos
<code>auth.test.ts</code>	Registro, login, perfil e troca de senha	9
<code>user.test.ts</code>	CRUD de usuários e permissões por papel	15
<code>product.test.ts</code>	CRUD de produtos, scan, filtros	8
<code>category.test.ts</code>	CRUD de categorias	9
<code>movement.test.ts</code>	Entrada/saída e atualização de saldo	5
<code>warehouse.test.ts</code>	Endereços de armazém	8
Total		54

```
npm run test          # suíte completa (--runInBand)
npm run test:watch    # modo watch
npm run test:coverage # relatório em coverage/
```

12 Operação e execução

● Execução local

```
cd backend
npm install
cp .env.example .env          # ajuste DATABASE_URL e JWT_SECRET
npm run prisma:migrate        # aplica migrations
npm run prisma:seed           # dados de demonstração
npm run dev                   # http://localhost:3001 (hot-reload)
```

● Variáveis de ambiente

Variável	Descrição
<code>DATABASE_URL</code>	String de conexão PostgreSQL
<code>PORT</code>	Porta HTTP (padrão 3001)
<code>JWT_SECRET</code>	Segredo de assinatura dos tokens
<code>JWT_EXPIRES_IN</code>	Validade do token (ex.: 7d)
<code>NODE_ENV</code>	development · test · production

● Scripts

Script	Função
<code>dev</code>	ts-node-dev com hot-reload
<code>build</code> · <code>start</code>	Compila para <code>dist/</code> e executa o build de produção
<code>prisma:migrate</code> · <code>prisma:generate</code>	Migrations e geração do client
<code>prisma:studio</code>	Interface gráfica do banco
<code>prisma:seed</code>	Popula usuário admin, catálogo e produto de exemplo
<code>test</code> · <code>test:coverage</code>	Testes e cobertura

● Documentação interativa

URL	Conteúdo
GET /	Painel HTML com todos os módulos e endpoints
GET /docs	Swagger UI com autenticação Bearer
GET /docs.json	Especificação OpenAPI 3.0

13 Infraestrutura em nuvem

A arquitetura-alvo do StockIQ roda na AWS, com a API e o banco em instâncias EC2 dentro de uma VPC, balanceamento por Application Load Balancer e serviços gerenciados de apoio.

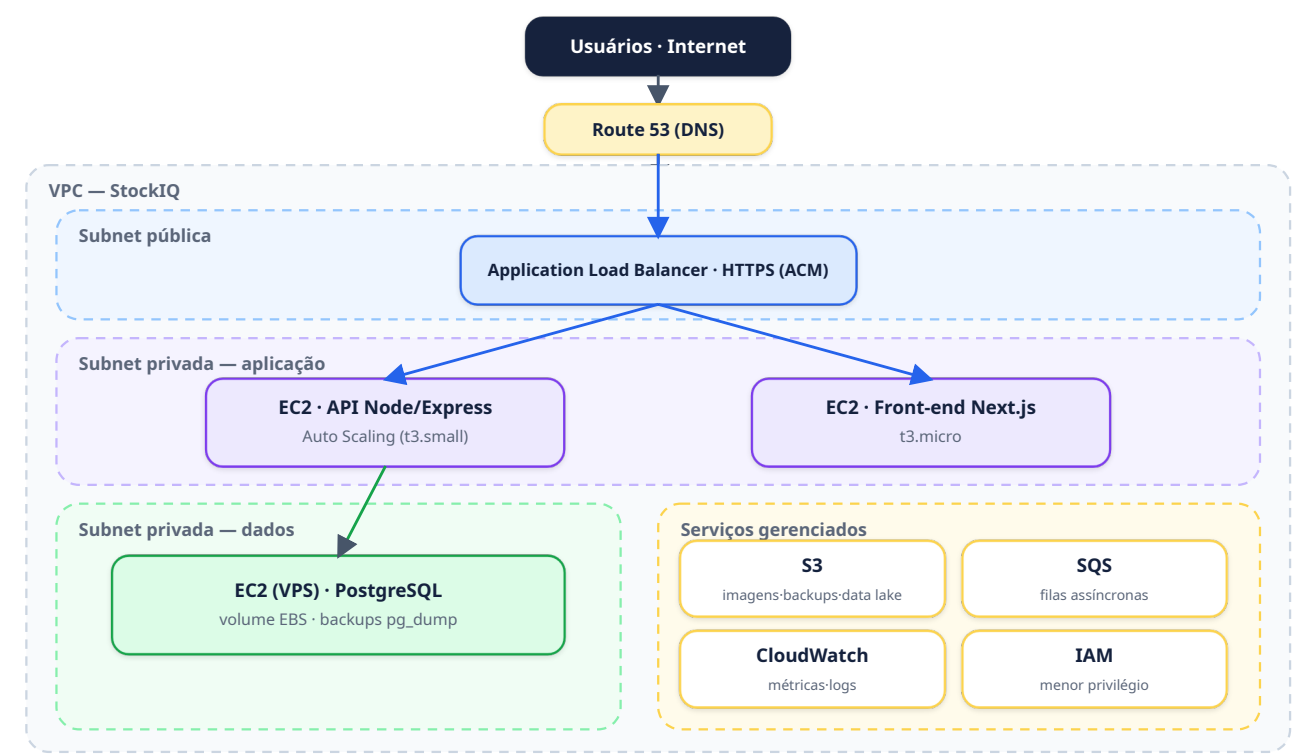


Figura 9 — Arquitetura de nuvem (AWS)

Serviço	Papel no sistema
EC2	Hospeda a API e, em instância separada, o PostgreSQL
Application Load Balancer	Distribui o tráfego, aplica health checks e habilita escala horizontal
VPC · Subnets · Security Groups	Somente o balanceador é público; API aceita tráfego só do ALB; banco só da API
S3	Imagens, backups agendados e data lake para mineração
SQS	Mensageria assíncrona (alertas, relatórios pesados, eventos de movimentação)
Route 53 · ACM	DNS e certificado TLS
CloudWatch · IAM	Observabilidade e permissões mínimas por instância

● Mensageria assíncrona (SQS)

1

Alertas e notificações

Eventos de estoque baixo/vencimento são publicados na fila e processados por um worker, sem atrasar a requisição.

2

Relatórios pesados

Curva ABC e exportações são enfileiradas e geradas em segundo plano.

3

Eventos para mineração

Cada movimentação publica um evento gravado no data lake (S3), isolando a análise do banco transacional.